Practical Assignment: Functional Testing, CI/CD and Live Deployment

Students receive `target.py` (16 functions: 11 data-type, 5 mathematics). Test it, host it on GitHub, automate it with GitHub Actions, add a frontend, and deploy it so every push is tested and goes live. Individual work, **20 points + 2 bonus**, due [date and time].

Do not modify `target.py`; the original is restored at grading. Report suspected bugs in your README.

Part 1: Functional tests (8 points)

Write `tests/test_target.py` that tests every corner case of all 16 functions.

- Cover valid inputs, boundary values, invalid values and wrong types for each function.
- Use `parametrize`, and `pytest.raises` with the exact exception class (plus `match=` where needed).
- Check result types where they matter; use `pytest.approx` only for inexact results.
- Reach 100% line and branch coverage: `pytest --cov=target --cov-branch`.
- No `skip`, `xfail` or other hidden failures.

Grading: corner cases 4 (0.25 per function), catching hidden faulty versions 2, precise assertions 1, coverage and quality 1.

Part 2: GitHub repository and sync (3 points + 2 bonus)

- Create `functional-testing-<student-id>` and give the instructor access.
- Make at least 10 focused commits with clear messages.
- Include `.gitignore` and a pinned `requirements.txt`; never commit secrets.
- Write a README with setup, test command, live URL, CI badge and suspected bugs.

Bonus: 3+ feature branches merged via pull requests (+1); branch protection on `main` requiring CI (+1).

Part 3: GitHub Actions workflow (3 points)

Create `.github/workflows/ci.yml` that:

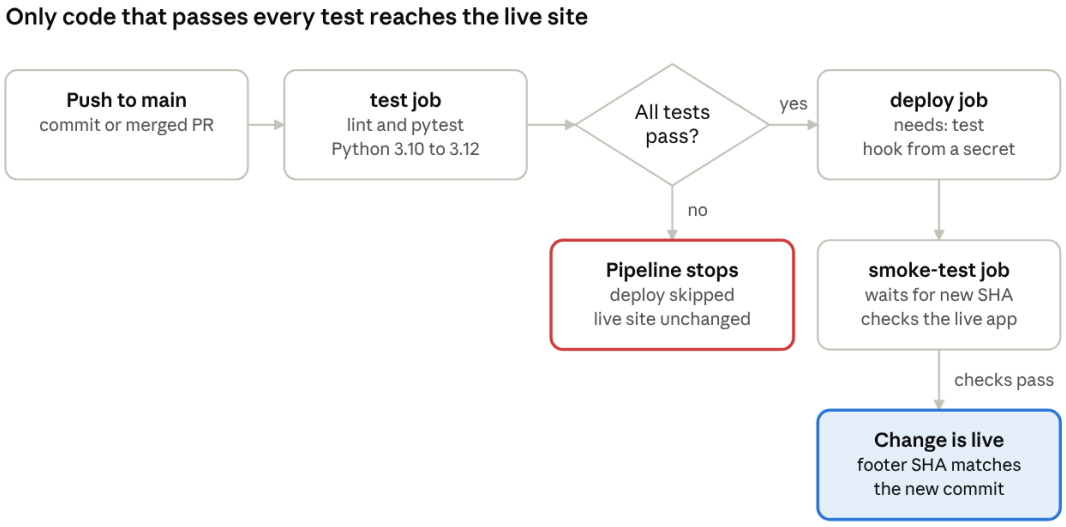
- runs on every push and on pull requests to `main`, in a job named `test`;
- tests on Python 3.10, 3.11 and 3.12 with pip caching;
- lints, runs all tests and fails below 100% coverage of `target.py`;
- uploads the coverage report as an artifact, with a status badge in the README.

Include one red run from a failing test, then a green fix.

Part 4: Frontend interaction (3 points)

Build a web frontend that runs the `target.py` functions and shows each result or error. Design and technology are your choice; instructors evaluate it directly.

Part 5: Deploy, integrate and test on push (3 points)



push-to-live pipeline · 3 jobs, 1 gate

- Deploy publicly (e.g. Render, Railway, PythonAnywhere) with direct auto-deploy off.
- Add a `deploy` job with `needs: test` that runs only on pushes to `main` and uses a GitHub secret.
- Make `/health` return the deployed commit SHA, and show it in the frontend.
- Add a `smoke-test` job that waits for the new SHA on the live site and checks three live requests.
- Record a 3–5 minute demo: a change going live with a matching SHA, and a failing push blocked from deploying.

Grading and submission

Part	Points
1. Functional tests	8
2. GitHub repository and sync	3 (+2 bonus)
3. GitHub Actions workflow	3
4. Frontend interaction	3
5. Deploy and test on push	3
Total	**20 + 2 bonus**

Submit the repository URL, live URL, demo video link and commit SHA to grade. Late policy: [policy].